

Reviewer Pack — Algebraic Hodge Index Bounds for Monotone Circuit Depth

Entry 4f93d04c05b9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 13:45:18 UTC

Algebraic Hodge Index Bounds for Monotone Circuit Depth

Entry ID: 4f93d04c05b9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 13:45:18 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): Monotone Boolean Circuit Complexity

Statement:

[‘For any monotone Boolean circuit C computing the OR of n variables, the algebraic Hodge index of the associated hypersurface, $h^1(C)$, is upper-bounded by a constant factor of the depth of C , $d(C)$.’, ‘Equivalently, for all monotone circuits C with depth $d(C)$ and input length n , there exists a constant c such that $h^1(C) \leq c * \log(d(C))$.’]

Rationale (proposer’s reasoning):

[‘The algebraic Hodge index measures the complexity of singularities in the associated algebraic variety. A deep circuit might be expected to have more complex singularities than a shallow one, suggesting a potential connection between Hodge theory and circuit depth.’, ‘If true, this conjecture would provide a new approach to proving lower bounds for monotone circuits by leveraging tools from real algebraic geometry.’]

Taxonomy category: AC0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 499271da4ada5ce8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a monotone Boolean circuit C , if the Hodge index $h^1(C)$ is less than or equal to $c * \log(d(C))$ for all circuits with depth $d(C)$, where c is a constant and $h^1(C) \leq 10,000$, the conjecture is supported. If any seed produces a metric $h^1(C) > 10,000$, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "algebraic Hodge index" AND "monotone Boolean circuit" - "real algebraic geometry" AND "circuit depth" AND "Hodge index bounds" - "depth of monotone circuits" AND "logarithmic bound" AND "Hodge index"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_monotone_circuit(n):
        circuit = ['OR']
        for _ in range(n):
            if random.choice([True, False]):
                circuit.append(['AND'])
            else:
                circuit.append(['NOT'])
            circuit.extend(generate_random_monotone_circuit(random.randint(1, 2)))
        return circuit

    def circuit_depth(circuit):
        if isinstance(circuit, list) and circuit[0] in ['OR', 'AND']:
            depths = [circuit_depth(sub_circuit) for sub_circuit in circuit[1:]]
            return max(depths) + 1
        return 0

    def hodge_index(circuit):
        # Placeholder function to simulate Hodge index calculation
        depth = circuit_depth(circuit)
        if depth == 0:
            return 0
        return Fraction(1, depth)

    n_values = [5, 10, 15, 20, 30, 40]
    hodge_indices = []
    depths = []
```

```

for n in n_values:
    circuit = generate_random_monotone_circuit(n)
    hodge_index_value = hodge_index(circuit)
    depth_value = circuit_depth(circuit)

    if hodge_index_value > 10000:
        return {
            "metric_name": "Hodge Index",
            "metric_value": hodge_index_value,
            "instances_tested": len(n_values),
            "conjecture_holds": False,
            "counterexample": f"Hodge index exceeds 10,000 for n={n}"
        }

    hodge_indices.append(hodge_index_value)
    depths.append(depth_value)

mean_hodge_index = sum(hodge_indices) / len(hodge_indices)
std_deviation = math.sqrt(sum((x - mean_hodge_index) ** 2 for x in hodge_indices) / len(hodge_indices))
support_fraction = all(x <= Fraction(10, depth) * math.log(depth) for x, depth in zip(hodge_indices, depths))

return {
    "metric_name": "Hodge Index",
    "metric_value": mean_hodge_index,
    "instances_tested": len(n_values),
    "conjecture_holds": support_fraction,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **trial_result}}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_dev = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    support_count = sum(1 for result in results if result["conjecture_holds"])

    if support_count == len(seeds):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction=1.0")
    elif support_count >= 0.8 * len(seeds):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_dev} support_fraction={support_count/len(seeds)}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Hodge index exceeds 10,000\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f852f19.py", line 83, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f852f19.py", line 49, in run_trial
    circuit = generate_random_monotone_circuit(n)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f852f19.py", line 28, in generate_rand
    circuit.extend(generate_random_monotone_circuit(random.randint(1, 2)))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f852f19.py", line 28, in generate_rand
    circuit.extend(generate_random_monotone_circuit(random.randint(1, 2)))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f852f19.py", line 28, in generate_rand
    circuit.extend(generate_random_monotone_circuit(random.randint(1, 2)))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

[Previous line repeated 993 more times]

```
File "/usr/lib/python3.12/random.py", line 336, in randint
    return self.randrange(a, b+1)
           ^^^^^^^^^^^^^^^^^^^^^
```

```
File "/usr/lib/python3.12/random.py", line 318, in randrange
    return istart + self._randbelow(width)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

RecursionError: maximum recursion depth exceeded

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a maximum recursion depth exceeded error before producing data that could confirm or falsify the conjecture. | next: Investigate and fix the recursion issue in the test code to allow it to complete and provide results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14395
2	preregistration	ollama_remote	glm4:latest	0	0	9586
3	novelty	ollama_remote	glm4:latest	0	0	8602
4	novelty	ollama_remote	glm4:latest	0	0	9067
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11459
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9605

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9684
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10262
9	judge	ollama_remote	glm4:latest	0	0	11532

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 94192 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/4f93d04c05b9.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/4f93d04c05b9.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4f93d04c05b9.tar.gz (if generated)